Protocol Workflow Analysis via Intensive Console Logging: Invariant Discovery Without Code Review

I started my next **First Flight at Cyfrin** - the protocol focuses on **reward distribution** among 'players'.

As a developer who spent countless hours debugging systems I didn't write (mostly .NET code), I've learned that a well-structured codebase can be invaluable for understanding the business logic, even when documentation is missing or outdated.

With that mindset, I dove into the tests for the smart contracts under audit to understand how the protocol behaves. The most interesting was the **test covering unclaimed reward distribution**, as it clearly reveals the system's internal accounting:

Here is a screenshot of that test with my blocks for logs (really intensive logging approach):

```
174     function testUnclaimedRewardDistribution() public mintAndApproveTokens {
175         vm.startPrank(user);
176         rewards = [500, 500];
177         totalRewards = 1000;
178         contest = ContestManager(conMan).createContest(players, rewards, IERC20(ERC20Mock(weth)), totalRewards);
179         console.log("                                                    ");
180         uint256 conManBalanceAfterCreateContest = ERC20Mock(weth).balanceOf(conMan);
181         console.log("conManBalanceAfterCreateContest ", conManBalanceAfterCreateContest.toString());
182
183         uint256 claimantBalanceAfterCreateContest = ERC20Mock(weth).balanceOf(player1);
184         console.log("claimantBalanceAfterCreateContest ", claimantBalanceAfterCreateContest.toString());
185
186         uint256 claimantBalanceAfterCreateContest2 = ERC20Mock(weth).balanceOf(player2);
187         console.log("claimantBalanceAfterCreateContest2 ", claimantBalanceAfterCreateContest2.toString());
188
189         uint256 remainingRewardsAfterCreateContest = Pot(contest).getRemainingRewards();
190         console.log("remainingRewardsAfterCreateContest ", remainingRewardsAfterCreateContest.toString());
191
192         ContestManager(conMan).fundContest(0);
193         vm.stopPrank();
194
195         console.log("-----------------------------");
196         uint256 conManBalanceBeforeClaim = ERC20Mock(weth).balanceOf(conMan);
197         console.log("conManBalanceBeforeClaim ", conManBalanceBeforeClaim.toString());
198
199         uint256 claimantBalanceBeforeClaim = ERC20Mock(weth).balanceOf(player1);
200         console.log("claimantBalanceBeforeClaim ", claimantBalanceBeforeClaim.toString());
201
202         uint256 claimantBalanceBeforeClaim2 = ERC20Mock(weth).balanceOf(player2);
203         console.log("claimantBalanceBeforeClaim2 ", claimantBalanceBeforeClaim2.toString());
204
205         uint256 remainingRewardsBeforeClaim = Pot(contest).getRemainingRewards();
206         console.log("remainingRewardsBeforeClaim ", remainingRewardsBeforeClaim.toString());
207
208         vm.startPrank(player1);
209         Pot(contest).claimCut();
210         vm.stopPrank();
211
212         console.log("-----------------------------");
213         uint256 conManBalanceAfterClaim = ERC20Mock(weth).balanceOf(conMan);
214         console.log("conManBalanceAfterClaim ", conManBalanceAfterClaim.toString());
215
216         uint256 claimantBalanceAfterClaim = ERC20Mock(weth).balanceOf(player1);
217         console.log("claimantBalanceAfterClaim ", claimantBalanceAfterClaim.toString());
218
219         uint256 claimantBalanceAfterClaim2 = ERC20Mock(weth).balanceOf(player2);
220         console.log("claimantBalanceAfterClaim2 ", claimantBalanceAfterClaim2.toString());
221
222         uint256 remainingRewardsAfterClaim = Pot(contest).getRemainingRewards();
223         console.log("remainingRewardsAfterClaim ", remainingRewardsAfterClaim.toString());
224         console.log("                                                    ");
225         vm.warp(91 days);
226
227         vm.startPrank(user);
228         ContestManager(conMan).closeContest(contest);
229         vm.stopPrank();
230
231         uint256 conManBalanceAfterCloseContest = ERC20Mock(weth).balanceOf(conMan);
232         console.log("conManBalanceAfterCloseContest ", conManBalanceAfterCloseContest.toString());
233
234         uint256 claimantBalanceAfterCloseContest = ERC20Mock(weth).balanceOf(player1);
235         console.log("claimantBalanceAfterCloseContest ", claimantBalanceAfterCloseContest.toString());
236
237         uint256 claimantBalanceAfterCloseContest2 = ERC20Mock(weth).balanceOf(player2);
238         console.log("claimantBalanceAfterCloseContest2 ", claimantBalanceAfterCloseContest2.toString());
239
240         uint256 remainingRewardsAfterCloseContest = Pot(contest).getRemainingRewards();
241         console.log("remainingRewardsAfterCloseContest ", remainingRewardsAfterCloseContest.toString());
242
243         assert(claimantBalanceAfterCloseContest > claimantBalanceAfterClaim);
244     }
245 }
246
```

To analyse this behaviour, I added **extensive logging**, capturing the state **after nearly every protocol call**.

```
-------------------------------
conManBalanceAfterCreateContest  0
claimantBalanceAfterCreateContest  0
claimantBalanceAfterCreateContest2  0
remainingRewardsAfterCreateContest  1000
-------------------------------
conManBalanceBeforeClaim  0
claimantBalanceBeforeClaim  0
claimantBalanceBeforeClaim2  0
remainingRewardsBeforeClaim  1000
-------------------------------
conManBalanceAfterClaim  0
claimantBalanceAfterClaim  500
claimantBalanceAfterClaim2  0
remainingRewardsAfterClaim  500
-----------------
conManBalanceAfterCloseContest  50
claimantBalanceAfterCloseContest  725
claimantBalanceAfterCloseContest2  0
remainingRewardsAfterCloseContest  500
```

I then compiled the logged outputs into a **table**, alongside my **questions and observations**, to make the protocol's behaviour easier to reason about.

At this stage, it's important to note that **I had not yet reviewed the smart contract implementation itself** - this analysis was based purely on observed behaviour.

```
address[] players = [player1, player2];
rewards           = [500, 500];
totalRewards      = 1000;

| flow              | conManBalance | player1bal | player2bal | remainingRewards |
| ----------------- | ------------- | ---------- | ---------- | ---------------- |
| create contest    | 0             | 0          | 0          | 1000             |
| fund contest      | 0             | 0          | 0          | 1000             |
| claimCut(palyer1) | 0             | 500        | 0          | 500              |
| 90 days passed    |               |            |            |                  |
| closeContest      | 50            | 725        | 0          | 500              |

1. from which balance weth are sent to player1 on claimCut?
2. why remaininRewards do not change after closeContest?
3. why player1 received +225 after closeContest - looks like (remainingRewards - conManBalance)  /  2
                                                                                why exactly 50?   why 2?
                                                                                                  after cut "the remainder is distributed equally to those
                                                                                                  who claimed in time!"
                                                                                                  -> 1 should be?
                                                                                                  -> where is the second half?
4. INVARIANT ? : `conManBalance + player1bal + player2bal + remainingRewards = totalRewards`
```

Based on these observations, I believe I have identified a **potential invariant in the system**. This hypothesis still needs to be validated through focused testing, and there's a chance my assumption is incorrect.

My next step is a manual review of the relevant smart contacts to confirm or disprove these findings.